

Traccia di soluzione

Modelli dati

Sala

```
{
  "idSala":      int:      3
  "nome":        str:      "aula alfa"
  "piano":       int:      1
  "capienza":    int:      30 // >0
  "dotazione":   List[str]: ["Proiettore", "Lavagna", ...]
}
```

Prenotazione

```
{
  "idPrenotazione": int:      1
  "idSala":          int:      3 // deve corrispondere a una sala esistente
  "idOrganizzatore": str:      "username"
  "data":            str:      "2026-05-08" // deve essere in formato YYYY-MM-DD
  "oralnizio":       str:      "10:30" // deve essere in formato HH:MM
  "oraFine":         str:      "10:40" // formato HH:MM ed essere successivo a oralnizio
  "titolo":          str:      "riunione"
  "dataCreazione":   str:      "2026-05-07" // deve essere in formato YYYY-MM-DD
}
```

Utente

```
{
  "idUtente":      str:      "username"
}
```

Domanda 1 – Progettazione delle API (15 punti)

API 1: Creazione prenotazione

Endpoint e metodo HTTP:

POST /prenotazioni

Modello di richiesta:

```
{
  #Sala -> fa riferimento all'oggetto definito prima
}
- idPrenotazione viene assegnato dal server alla creazione della prenotazione e restituito in risposta
- idSala deve corrispondere a una sala esistente
- idUtente è passato automaticamente dal server in quanto l'utente risulta loggato
```

Modello di risposta (201 Created o 200 OK):

```
{
  "idPrenotazione" : int : 77
}
```

API 2: Sale disponibili per data e orario

Endpoint e metodo HTTP:

GET /sale?data={data}&oralnizio={oralnizio}&oraFine={oraFine}

I tre parametri sono obbligatori per la ricerca disponibilità. Senza parametri: lista completa di tutte le sale.

Esempio: GET /sale?data=2026-11-10&oraInizio=10:00&oraFine=11:30

Modello di risposta (200 OK):

- ```
[{ idSala:3, nome:"Sala Blu", piano:2, capienza:10,
 dotazione:["proiettore","lavagna"] }, { ... }]
```
- in questo caso la logica del server dovrebbe fare la verifica delle prenotazioni associate a ogni sala e restituire la sala nella lista se nessuna delle prenotazioni associate all'idSala selezionato è tra l'ora inizio e ora fine indicati per il giorno

### **API 3: Aggiunta partecipante**

#### **Endpoint e metodo HTTP:**

POST /prenotazioni/{idPrenotazione}/partecipanti

#### **Modello di richiesta:**

- ```
POST /prenotazioni/77/partecipanti
{ idPartecipante : int : 9 // obbligatorio, deve corrispondere a un utente esistente }
```
- l'idPrenotazione è già incluso nel path parameter, quindi non sarà incluso nel JSON (per evitare inconsistenze). Il server sa già a che prenotazione associare la richiesta perché la POST arriva all'url associato.
 - in questo caso si può anche assumere che un utente possa aggiungere altri utenti (non solo se stesso) alla prenotazione. In questo caso si lascia libertà di scelta – nel caso va specificato che "idUtente" è passato dal login.
 - La prenotazione andrà a guardare se la lista di utenti registrati per la prenotazione è superiore alla capienza. In caso non ci sia posto, restituirà un codice di errore.
 - A questo punto si può prevedere di aggiungere il parametro "partecipanti": List[str] che contiene la lista di partecipanti iscritti (o alternativamente un contatore che viene incrementato)

Modello di risposta (201 Created o 200 OK):

```
{ idPrenotazione:77, idPartecipante:9, dataAggiunta:"2026-11-05" }
```

Logica di controllo e vincoli aggiuntivi (ne bastano 3):

Se sono già stati scritti sopra non c'è bisogno di specificarli di nuovo. Li aggiungo qui solo per completezza.

- Controllo sovrapposizione oraria: prima di confermare una prenotazione il server verifica che non esista già una prenotazione per la stessa sala nella stessa data con orari sovrapposti (422 altrimenti).
- Controllo capienza: il numero totale di partecipanti aggiunti non può superare la capienza della sala (422).
- Validazione orari: oraFine deve essere strettamente successiva a oraInizio.
- Controllo esistenza sala: idSala deve corrispondere a una sala registrata (404 altrimenti).

Domanda 2 – Validazione e gestione errori (5 punti)

Descrizione codici di errore:

404 Not Found: la risorsa identificata dall'ID fornito non esiste nel sistema. Viene restituito quando il client tenta di accedere a un'entità non presente nel database.

422 Unprocessable Entity: la richiesta è sintatticamente corretta ma contiene dati che violano i vincoli logici o di validazione dell'applicazione (es. valori fuori range, campi non consentiti, violazioni di regole di business).

500 Internal Server Error: errore interno non previsto, indipendente dall'input del client. Indica un malfunzionamento del sistema, come un database irraggiungibile o un'eccezione non gestita.

Esempio di errore 404:

Tentativo di aggiungere un partecipante a una prenotazione inesistente:

```
POST /prenotazioni/9999/partecipanti
HTTP/1.1 404 Not Found
{ "error": "Booking not found", "message": "La prenotazione con ID 9999 non esiste." }
```

Esempio di errore 422:

Prenotazione con orari sovrapposti a una esistente:

```
POST /prenotazioni { "idSala":3, "data":"2026-11-10", "oraInizio":"10:30", "oraFine":"12:00", ... }
HTTP/1.1 422 Unprocessable Entity
{ "error": "Time slot conflict",
  "message": "La sala è già prenotata dalle 10:00 alle 11:30 in questa data." }
```

Domanda 3 – Estensione (3 punti)

Traccia di risposta:

- Modello dati: aggiungere alla prenotazione i campi ricorrente (bool), frequenza ("settimanale"/"bisettimanale"), numOccorrenze (int, min=2, max=52) e idPrenotazioneBase (per collegare le occorrenze alla serie). Il server genera automaticamente tutte le occorrenze.
- Logica server-side: alla creazione di una prenotazione ricorrente il server controlla la disponibilità per ciascuna occorrenza. Se una o più occorrenze sono in conflitto, il server può restituire un 422 con l'elenco delle date problematiche, permettendo all'organizzatore di escluderle o di scegliere un orario alternativo.